

Command Reference

Cylops Studio operations bot — every command, who can use it, and what it does

How to read the "who can use it" column

The badge is the plain-language answer. The grey text under it, where there is any, is the exact capability the code checks — the same name you would type into `/studio capability`.

These are defaults, not fixed rules. Nothing in the bot checks Discord roles directly. Roles are mapped to capabilities in configuration, so the studio can restructure its roles without a code change. If you grant `payment.record` to a leader role, that leader can record payments and this page's "Owner" badge no longer describes your studio.

The owner always holds every capability. A group leader holds their leader capabilities *only inside a department they lead* — the same command refuses them in somebody else's department. Someone standing in for a leader gets the same powers, in that department, until the stand-in date passes.

Two things worth knowing before you read on

Commands you cannot use are not listed in `/help`, and most replies are private to you. Where a command shows money, it shows it only to people who could already see it.

The bot records that money moved. **It never moves money**, and it never asks for a payment password, card number or wallet phrase.

Commands at a glance

Grouped by the easiest way in. A `starred *` command has subcommands above that level too — its own heading gives the range, and the table gives each subcommand exactly. The other 15 are the same the whole way through.

● Anyone on the team can use at least part of these

`/bonuses *` `/concern *` `/deadlines *` `/desk *` `/files *` `/find` `/go` `/help` `/messages *` `/my-work *` `/orders *` `/profile *` `/staff-rules *` `/task *` `/time` `/trial *`

● The person doing the work can use at least part of these

`/contrib *`

● Group leader can use at least part of these

`/change *` `/problems` `/recommend *` `/review *` `/send *` `/who-is-free`

● Manager can use at least part of these

`/clients` `/quotes *` `/reorder` `/reports *` `/studio *` `/summary *`

● Owner can use at least part of these

[/automation](#) [/backup](#) [/money-in](#) [/pay](#) [/people](#) [/proof](#) [/web](#)

36 commands · 222 subcommands · 4 commands that take no subcommand · generated 2026-09-30 from the command definitions in `src/commands`. Rebuild with `npm run reference`.

/automation Owner

Rules that watch for a situation and tell somebody

COMMAND	WHO CAN USE IT	WHAT IT DOES
/automation set key: label: when: then: [days:] [department:] [person:] [note:]	Owner config.manage	Create or change a rule (it starts switched off)
/automation list	Owner config.manage	Every rule, and whether it is on
/automation preview key:	Owner config.manage	What a rule would do right now — it does none of it
/automation on key:	Owner config.manage	Switch a rule on
/automation off key:	Owner config.manage	Switch a rule off
/automation delete key:	Owner config.manage	Remove a rule entirely

/backup Owner

Backups you can restore from, and exports you can read

COMMAND	WHO CAN USE IT	WHAT IT DOES
/backup now	Owner config.manage + finance.view_all	Take a consistent copy of the database, right now
/backup export what:	Owner config.manage + finance.view_all	Download readable data as a spreadsheet file
/backup verify [file:]	Owner config.manage + finance.view_all	Check a backup file is really a database that opens
/backup restore	Owner config.manage + finance.view_all	How to put a backup back — read this before you need it

/bonuses Anyone on the team – Owner

Milestone bonuses: rules, what has been earned, and approvals

COMMAND	WHO CAN USE IT	WHAT IT DOES
/bonuses rule-set key: label: threshold: amount: [currency:] [department:]	Owner task.pay.approve	Create or change a bonus rule
/bonuses rules	Manager or owner summary.view	Every bonus rule in the studio

COMMAND	WHO CAN USE IT	WHAT IT DOES
/bonuses rule-off key:	Owner task.pay.approve	Stop a rule earning anything further (past awards are kept)
/bonuses rule-on key:	Owner task.pay.approve	Put a rule back into use
/bonuses pending	Owner task.pay.approve	Milestones reached and waiting on you
/bonuses approve id:	Owner task.pay.approve	Agree a bonus is owed
/bonuses decline id: reason:	Owner task.pay.approve	Decide a bonus is not owed, with a reason
/bonuses pay id: [method:] [reference:]	Owner task.pay.approve	Record an approved bonus as paid
/bonuses mine	Anyone	Your own bonus progress and awards

/change **Group leader** – **Owner**

Reassign, hold, cancel and compensate

COMMAND	WHO CAN USE IT	WHAT IT DOES
/change reassign task: artist: reason: [handoff:]	Group leader (own department) task.reassign	Move a task to a different artist, keeping the original record
/change hold task: reason:	Group leader (own department) task.hold	Put a task on hold
/change resume task:	Group leader (own department) task.hold	Take a task off hold
/change cancel task: reason:	Owner task.cancel	Cancel a task, preserving its history
/change compensate task: member: amount: [currency:] [reason:] [method:]	Owner payment.record	Record a payment for work done on a cancelled or reassigned task
/change flags	Manager or owner summary.view	Tasks waiting on a decision from you

/clients **Manager**

Client records, access and their dashboard

COMMAND	WHO CAN USE IT	WHAT IT DOES
/clients create name: [account:] [finder:] [notes:]	Manager or owner project.edit	Create a client record
/clients add-account client: account: [can_approve:] [label:]	Manager or owner project.edit	Authorize a Discord account to act for a client

COMMAND	WHO CAN USE IT	WHAT IT DOES
/clients revoke-account client: account:	Manager or owner project.edit	Remove an account's access
/clients link project: client: [channel:]	Manager or owner project.edit	Attach a project to a client and their channel
/clients dashboard project:	Manager or owner project.edit	Post or refresh a project's client dashboard
/clients view client:	Manager or owner project.edit	A client record, their accounts and order history
/clients list	Manager or owner project.edit	All client records
/clients preview project:	Manager or owner project.edit	See exactly what a client can see on a project
/clients requests [status:]	Manager or owner project.edit	Open client requests and questions
/clients resolve id: status: [note:] [tell_client:]	Manager or owner project.edit	Close a client request with a note
/clients duplicates	Manager or owner project.edit	Possible duplicate client records
/clients requirements client:	Manager or owner project.edit	Standing requirements this client has on every order
/clients add-requirement client: label: detail: [department:]	Manager or owner project.edit	Record something this client always wants
/clients retire-requirement id:	Manager or owner project.edit	Stop showing a standing requirement (it is kept on the record)

/concern Anyone on the team – Owner

Raise a concern privately, or handle raised concerns

COMMAND	WHO CAN USE IT	WHAT IT DOES
/concern raise category: subject: details: [task:]	Anyone	Raise a concern about an assignment, your pay or your workload
/concern mine	Anyone	Concerns you have raised and where they stand
/concern list [status:]	Owner config.manage	Concerns raised by staff (owner only)
/concern respond id: action: [note:]	Owner config.manage	Acknowledge or resolve a raised concern (owner only)

/contrib The person doing the work – Group leader

People sharing one task, and what each of them is paid

COMMAND	WHO CAN USE IT	WHAT IT DOES
/contrib add task: person: responsibility:	Group leader (own department) task.edit	Add somebody to a task alongside the main artist
/contrib remove task: person:	Group leader (own department) task.edit	Take somebody off a task (their record and any payments stay)
/contrib pay task: person: amount: [currency:]	Leader proposes, owner sets	Set one contributor's pay (owner) or propose it (group leader) <i>A leader can put a figure forward. Only the owner makes it the agreed pay.</i>
/contrib list task:	Leader or owner sees all; others their own	Who is on a task, and what each is owed <i>Everybody's figures on a shared task are a leader-and-above view.</i>

/deadlines Anyone on the team – Manager

Templates, dependencies, blockers and deadline changes

COMMAND	WHO CAN USE IT	WHAT IT DOES
/deadlines template key: department: label: [brief:] [deliverables:] [formats:] [tech:] [revisions:] [days:]	Manager or owner task.create	Save a reusable task template
/deadlines templates	Anyone	Saved task templates
/deadlines batch project: template: count: [deadline:]	Manager or owner task.create	Create several tasks from a template
/deadlines depends task: needs: [note:]	Group leader (own department) task.edit	Record that one task needs another finished first
/deadlines undepend task: needs:	Group leader (own department) task.edit	Remove a dependency
/deadlines chain task:	Anyone	Show what a task waits on and what waits on it
/deadlines risks	Anyone — results filtered	Work whose prerequisite is late <i>Returns only what you already have access to, so a result cannot reveal that something exists.</i>
/deadlines ready task:	Anyone	Check whether a task is complete enough to offer
/deadlines blockers	Anyone — results filtered	Open blockers across your departments <i>Returns only what you already have access to, so a result cannot reveal that something exists.</i>
/deadlines clear-blocker id: [note:]	Whoever raised it, their leader, or the owner	Mark a blocker resolved <i>The person who hit the blocker usually knows first that it is gone.</i>

COMMAND	WHO CAN USE IT	WHAT IT DOES
/deadlines extend task: new_deadline: reason:	The artist, or their leader	Request a later deadline for a task <i>An artist can ask for more time on their own work; a leader can change the date.</i>
/deadlines extensions	Anyone — results filtered	Deadline requests waiting on a decision <i>Returns only what you already have access to, so a result cannot reveal that something exists.</i>
/deadlines decide-extension id: approve: [note:]	Group leader (own department) task.edit	Approve or decline a deadline request

/desk Anyone on the team – Manager

Your work in one place

COMMAND	WHO CAN USE IT	WHAT IT DOES
/desk me	Anyone	Your offers, assignments, deadlines and pay
/desk group [department:]	Group leader (own department)	Your department: queue, capacity, reviews and risks <i>The owner sees every department; a leader sees the ones they lead.</i>
/desk owner	Manager or owner summary.view	Everything waiting on the studio owner
/desk web-link	Anyone	A one-time link to open your desk in a browser

/files Anyone on the team – Manager

Search past work and manage portfolio permission

COMMAND	WHO CAN USE IT	WHAT IT DOES
/files search [query:] [project:] [artist:] [type:] [kind:] [portfolio:]	Anyone on the team	Find files across past work
/files rights project: staff: studio: [from:] [restrictions:]	Manager or owner project.edit	Record what the client allows for portfolio use
/files asset-rights asset: staff: studio: [from:] [restrictions:]	Manager or owner project.edit	Override portfolio permission for one file
/files portfolio	Anyone on the team	What the studio may publish right now
/files roblox-id asset: id:	Manager or owner project.edit	Record the Roblox asset ID a file was uploaded as
/files roblox-ids project:	Anyone on the team	Every Roblox asset ID recorded on an order

/find Anyone on the team

Search everything at once — work, orders, clients, people, files

COMMAND	WHO CAN USE IT	WHAT IT DOES
/find query: [only:]	Anyone — results filtered	Search everything at once — work, orders, clients, people, files <i>Returns only what you already have access to, so a result cannot reveal that something exists.</i>

/go Anyone on the team

What is waiting on you right now

COMMAND	WHO CAN USE IT	WHAT IT DOES
/go	Anyone	What is waiting on you right now

/help Anyone on the team

What you can do with this bot

COMMAND	WHO CAN USE IT	WHAT IT DOES
/help	Anyone	What you can do with this bot

/messages Anyone on the team – Owner

Automated client messages: templates, queue and history

COMMAND	WHO CAN USE IT	WHAT IT DOES
/messages template-set key: label: body: kind: [trigger:]	Owner or client manager client.record	Write or rewrite a message template (it starts as a draft)
/messages template-approve key:	Owner client.record + config.manage	Approve a template so it can actually send
/messages templates	Owner or client manager client.record	Every template and whether it is approved
/messages preview key:	Owner or client manager client.record	See a template with sample values, without sending anything
/messages placeholders	Anyone	Every value a template may use
/messages queue	Owner or client manager client.record	Messages waiting to go out, and ones that were held back
/messages cancel id:	Owner or client manager client.record	Cancel a queued message before it sends
/messages history client:	Owner or client manager client.record	What has been sent to a client

COMMAND	WHO CAN USE IT	WHAT IT DOES
/messages prefs client: [max-per-week:] [pause-days:] [pause-reason:] [follow-up:] [promotional:]	Owner or client manager client.record	Set a client's messaging rules
/messages replies	Owner or client manager client.record	Clients who wrote in and are waiting on an answer
/messages handled id:	Owner or client manager client.record	Mark a client reply dealt with, which lets automation resume
/messages offer project:	Owner or client manager client.record	Queue the approved cross-service offer for one order

/money-in **Owner**

Split what a client owes into named parts, and see what has arrived

COMMAND	WHO CAN USE IT	WHAT IT DOES
/money-in add project: label: amount: [currency:] [due:]	Owner client_receipt.record	Add a part of what the client owes, e.g. "Deposit" or "On delivery"
/money-in list project:	Owner finance.view_all	The parts on an order, and which are covered
/money-in invoiced id:	Owner client_receipt.record	Record that you have asked the client for this part
/money-in waive id: reason:	Owner client_receipt.record	Decide not to charge a part — it stops counting as owed
/money-in remove id:	Owner client_receipt.record	Delete a part that was entered by mistake

/my-work **Anyone on the team** – **The person doing the work**

Post progress and submit finished work

COMMAND	WHO CAN USE IT	WHAT IT DOES
/my-work progress task: note: [links:]	The assigned artist	Post a progress update on a task you hold <i>Your own action on your own task. Nobody reports progress or submits on somebody else's behalf.</i>
/my-work submit task:	The assigned artist	Submit finished work for internal review <i>Your own action on your own task. Nobody reports progress or submits on somebody else's behalf.</i>
/my-work history task:	The artist, or their leader	Submissions, reviews and client decisions on a task <i>Your own work, or work in a department you lead.</i>
/my-work blocked task: reason: [link:]	The assigned artist	Say you cannot continue, and why <i>Your own action on your own task. Nobody reports progress or submits on somebody else's behalf.</i>

COMMAND	WHO CAN USE IT	WHAT IT DOES
/my-work unblock task: [note:]	The assigned artist	Say a blocker you raised is resolved <i>Your own action on your own task. Nobody reports progress or submits on somebody else's behalf.</i>
/my-work earnings	Anyone — their own only	Your own pay and payment history (private) <i>Shows you your own figures. Nobody sees anybody else's pay through it.</i>

/orders Anyone on the team – Manager

Client projects and their tasks

COMMAND	WHO CAN USE IT	WHAT IT DOES
/orders create name: [client_ref:] [budget:] [currency:] [deadline:] [manager:] [finder:] [mod:] [ticket:]	Manager or owner project.create	Create a project
/orders view project:	Anyone — money hidden	Show a project <i>Prices and pay appear only if you could see them anyway (finance.view_all or owner).</i>
/orders list [status:]	Anyone on the team	List projects
/orders tasks project:	Anyone — money hidden	List the tasks on a project <i>Prices and pay appear only if you could see them anyway (finance.view_all or owner).</i>
/orders bulk project: spec: [deadline:]	Manager or owner task.create	Turn a bulk order into tasks, e.g. "12 models, 4 vfx, 2 animations"
/orders edit project: [brief:] [references:] [budget:] [currency:] [ticket:] [finder:] [mod:] [status:]	Manager or owner project.edit	Change a project field

/pay Owner

Client receipts, staff payouts and the studio ledger

COMMAND	WHO CAN USE IT	WHAT IT DOES
/pay client-receipt project: amount: [currency:] [method:] [reference:]	Owner client_receipt.record	Record money received from a client
/pay pay task: [person:] [amount:] [method:] [reference:]	Owner payment.record	Record a payout to someone who worked on the task
/pay pay-split task: share: [amount:] [method:] [reference:]	Owner payment.record	Record a payout of a finder, leader, mod or owner share
/pay splits task:	Owner finance.view_all	Show how a task's leftover pool divides

COMMAND	WHO CAN USE IT	WHAT IT DOES
/pay set-pool task: amount: currency:	Owner payment.record	Enter the distributable pool by hand when currencies differ
/pay mark-payable task: reason:	Owner payment.record	Make approved work payable before the client payment lands
/pay ledger [project:]	Owner finance.view_all	Money in and money out, per currency
/pay outstanding	Owner finance.view_all	Approved work that still owes somebody money
/pay balance member:	Owner finance.view_all	What a staff member is owed and has been paid
/pay budget project:	Owner finance.view_all	What a project has committed in pay against what the client pays
/pay budget-override project: reason:	Owner finance.view_all + payment.record	Deliberately allow pay above the recorded client payment on a project
/pay budget-restore project:	Owner finance.view_all + payment.record	Put the budget guard back on a project
/pay confirm-received payment: [note:]	Owner payment.record	The person says the money actually reached them
/pay mark-failed payment: reason:	Owner payment.record	A payment was sent and did not arrive — they are owed it again
/pay sent	Owner finance.view_all	Payments sent that nobody has confirmed arrived
/pay approve-all [project:] [preview:]	Owner task.pay.approve	Approve every pay figure your leaders have proposed

/people **Owner**

Stand-in leaders and offboarding

COMMAND	WHO CAN USE IT	WHAT IT DOES
/people stand-in grant person: department: until: responsibilities: [from:]	Owner staff.manage	Let somebody run a department for a stated period
/people stand-in revoke id:	Owner staff.manage	End a stand-in arrangement early
/people stand-in list [include-past:]	Owner staff.manage	Stand-in arrangements
/people offboard preview person:	Owner staff.manage	What they would leave behind — changes nothing
/people offboard start person: [reason:] [handover:] [withdraw-offers:]	Owner staff.manage	Record the departure and take them off the boards

COMMAND	WHO CAN USE IT	WHAT IT DOES
/people offboard complete id:	Owner staff.manage	Mark an offboarding finished once nothing is outstanding
/people offboard list	Owner staff.manage	Departures, open ones first

/problems **Group leader**

Problems reported by clients

COMMAND	WHO CAN USE IT	WHAT IT DOES
/problems list [status:]	Manager, or the department it is against	Reported problems <i>You handle problems reported against work your department did.</i>
/problems view id:	Manager, or the department it is against	One reported problem in full <i>You handle problems reported against work your department did.</i>
/problems decide id: decision: note: [tell_client:]	Leader decides revisions, owner decides money	Judge whether a reported problem is in scope or extra work <i>A free fix is a leader's call. Anything that costs money is the owner's.</i>
/problems close id: note: [tell_client:]	Manager, or the department it is against	Close a reported problem <i>You handle problems reported against work your department did.</i>

/profile **Anyone on the team** – **Owner**

Your studio profile: timezone, availability, specialties and hours

COMMAND	WHO CAN USE IT	WHAT IT DOES
/profile me	Anyone	Open your profile to fill in or edit it
/profile view member:	Anyone	View a staff member's profile
/profile timezone timezone:	Anyone	Set your timezone
/profile availability status: [until:] [note:]	Anyone	Say whether you are accepting work (separate from being online)
/profile assign member: department:	Owner staff.manage	Put a staff member in a department (managers only)

/proof **Owner**

Keep screenshots as proof, and produce the full record of an order

COMMAND	WHO CAN USE IT	WHAT IT DOES
/proof add file: kind: [note:] [project:] [task:] [client:]	Owner or client manager client.record	File a screenshot or document as proof
/proof list [project:] [task:]	Owner or client manager client.record	What has been filed

COMMAND	WHO CAN USE IT	WHAT IT DOES
/proof show id:	Owner or client manager client.record	Get a filed file back
/proof verify	Owner or client manager client.record	Check every filed file is still the one that was filed
/proof record project:	Owner client.record + finance.view_all	The whole record of an order, as a file — for a dispute or a chargeback

/quotes Manager – Owner

Incoming work enquiries and quotes

COMMAND	WHO CAN USE IT	WHAT IT DOES
/quotes new service: [client:] [contact:] [references:] [formats:] [deadline:] [budget:] [notes:]	Manager or owner project.create	Record an enquiry that came in outside the bot
/quotes list [status:]	Manager or owner project.create	The enquiry pipeline
/quotes view enquiry:	Manager or owner project.create	One enquiry and its quotes
/quotes needs-info enquiry: question:	Manager or owner project.create	Mark that you are waiting on the client for details
/quotes draft-quote enquiry: [total:] [currency:] [turnaround_days:] [terms:]	Manager or owner project.create	Prepare a draft quote from your templates (still needs owner approval)
/quotes approve-quote enquiry:	Owner task.pay.approve	Approve a draft quote so it can be sent (owner only)
/quotes send-quote enquiry: [post_to:]	Manager or owner project.create	Record that the approved quote has been sent to the client
/quotes accept enquiry: [deadline:] [client_channel:]	Manager or owner project.create	Client accepted: turn the enquiry into a project with its tasks
/quotes decline enquiry: reason:	Manager or owner project.create	Record that the enquiry did not go ahead
/quotes template department: unit_price: [currency:] [turnaround_days:] [revisions:]	Owner task.pay.approve	Set a pricing template for a service
/quotes templates	Manager or owner project.create	Current pricing templates

/recommend Group leader – Owner

Put somebody forward for a trial, a promotion or a leader role

COMMAND	WHO CAN USE IT	WHAT IT DOES
/recommend new person: kind: note: [department:]	Leader or manager	Recommend somebody <i>Putting somebody forward is a leader's job; deciding is the owner's.</i>
/recommend list [status:]	Owner staff.manage	Recommendations waiting on a decision
/recommend decide id: decision: [note:]	Owner staff.manage	Accept or decline a recommendation

/reorder Manager

Order a client something like last time

COMMAND	WHO CAN USE IT	WHAT IT DOES
/reorder from project: [name:]	Manager or owner project.create	Start a draft from a previous order
/reorder show id:	Manager or owner project.create	See a draft and what is still to confirm
/reorder list	Manager or owner project.create	Drafts waiting to be confirmed
/reorder scope id: [spec:]	Manager or owner project.create	Confirm what is in the order, changing it if needed
/reorder price id: amount: [currency:]	Manager or owner project.create	Confirm what the client is paying this time
/reorder deadline id: date:	Manager or owner project.create	Confirm the deadline for this order
/reorder create id: [channel:]	Manager or owner project.create	Turn a fully confirmed draft into a real order
/reorder discard id:	Manager or owner project.create	Throw a draft away

/reports Manager – Owner

Filters and reports over the studio's own records

COMMAND	WHO CAN USE IT	WHAT IT DOES
/reports filter which: [department:]	Manager or owner summary.view	Work matching one plain question
/reports overview	Manager or owner summary.view	How many tasks match each filter right now
/reports waiting project:	Manager or owner summary.view	Where an order's time has gone — with the studio, or with the client

COMMAND	WHO CAN USE IT	WHAT IT DOES
/reports person member: [days:]	Manager or owner summary.view	A rounded picture of one person's work — not a score
/reports team	Manager or owner summary.view	Everybody's picture, in name order
/reports payouts	Owner summary.view + finance.view_all	Everyone owed money on approved work
/reports attention	Manager or owner summary.view	Orders that are overdue or have stalled with us

/review Group leader – Owner

Internal review and recording client decisions

COMMAND	WHO CAN USE IT	WHAT IT DOES
/review queue [department:]	Group leader (own department) review.internal	Work waiting for your internal review
/review decide task:	Group leader (own department) review.internal	Review one task: request changes or mark it ready for the client
/review client task: decision: [feedback:] [reference:] [out_of_scope:]	Owner or client manager	Record the client's decision after they replied to you <i>Recording what a client decided is a statement about the client, so it is not a leader's call by default.</i>
/review awaiting-client	Leader, manager or owner	Work that has been sent to clients and has no decision yet

/send Group leader – Owner

Release finished work to a client

COMMAND	WHO CAN USE IT	WHAT IT DOES
/send check task:	Manager, reviewer or owner	See whether an item is ready to release, and what is blocking it
/send release task: [note:] [override:]	Manager or owner	Authorize delivery of an item to the client <i>Releasing against unmet conditions additionally needs client.record, so an override is always a deliberate act.</i>
/send pending	Manager or owner project.edit	Approved work that has not been released yet
/send conditions [require_client_approval:] [require_client_paid:] [require_checklist_complete:]	Owner config.manage	What must be true before work can be released

/staff-rules Anyone on the team – Owner

Studio procedures and acknowledgements

COMMAND	WHO CAN USE IT	WHAT IT DOES
/staff-rules list	Anyone	Procedures that apply to you, and which you still owe
/staff-rules read key:	Anyone	Read one, with a button to acknowledge it
/staff-rules set key: title: body: [audience:] [department:]	Owner config.manage	Create a procedure, or replace its text
/staff-rules retire key:	Owner config.manage	Stop showing a procedure (its acknowledgements are kept)
/staff-rules who key:	Owner staff.manage	Who has acknowledged the current version, and who has not

/studio Manager – Owner

Configure the studio bot (owner only)

COMMAND	WHO CAN USE IT	WHAT IT DOES
/studio setup	Owner, or whoever sets it up first	First-run setup: claim ownership and create the default departments <i>Before an owner is recorded, anybody who can manage the server may claim the studio once. After that it needs config.manage.</i>
/studio status	Owner config.manage	Show the current configuration
/studio doctor	Manager or owner summary.view	Find what is quietly misconfigured or stuck
/studio channel purpose: channel:	Owner config.manage	Set one of the bot channels
/studio roles [owner:] [manager:]	Owner config.manage	Set the owner and manager roles
/studio department key: [name:] [leader_role:] [member_role:] [task_cap:] [checklist:]	Owner config.manage	Create or update a department and its role mapping
/studio departments	Owner config.manage	List departments and their role mappings
/studio splits finder: leader: mod: owner:	Owner config.manage	Set how the leftover pool is divided (must total 100%)
/studio reminders [board_refresh_minutes:] [offer_reminder_hours:] [stale_progress_days:] [deadline_warning_hours:] [quiet_start:] [quiet_end:]	Owner config.manage	Reminder timings and studio-wide quiet hours

COMMAND	WHO CAN USE IT	WHAT IT DOES
/studio currency currency:	Owner config.manage	Set the default currency for new projects
/studio capability mode: role: capability:	Owner config.manage	Grant or revoke a capability for a role
/studio capabilities	Owner config.manage	Show which roles hold which capabilities
/studio refresh	Owner config.manage	Rebuild the staff boards now
/studio sample action: [leader:] [artist:]	Owner config.manage	Create or remove a demonstration project showing the whole workflow

/summary **Manager** – **Owner**

Management summary and reminder controls

COMMAND	WHO CAN USE IT	WHAT IT DOES
/summary now	Manager or owner summary.view	Show the weekly summary right now, privately
/summary post	Manager or owner summary.view	Post the weekly summary to its channel now
/summary schedule cron:	Owner summary.view + config.manage	Change when the weekly summary is posted
/summary run-reminders	Owner summary.view + config.manage	Run the reminder sweep now instead of waiting for the next one

/task **Anyone on the team** – **Owner**

Tasks, assignment and pay

COMMAND	WHO CAN USE IT	WHAT IT DOES
/task create project: department: title: [brief:] [deadline:] [formats:] [tech:] [references:] [revisions:]	Manager or owner task.create	Create a single task
/task view task:	Your task, your department, or owner	Show a task <i>You see tasks assigned to you, tasks in a department you lead, or everything if you hold finance access.</i>
/task queue [department:]	Group leader (own department) task.offer	Unassigned tasks waiting for you to pick an artist
/task assign task:	Group leader (own department) task.offer	Choose the artist for a task and send them the offer
/task pay task: amount: [currency:]	Leader proposes, owner sets	Set pay (owner) or propose it (group leader) <i>A leader can put a figure forward. Only the owner makes it the agreed pay.</i>

COMMAND	WHO CAN USE IT	WHAT IT DOES
/task approve-pay task: [amount:] [currency:]	Owner task.pay.approve	Approve a leader's proposed pay (owner)
/task edit task: [title:] [brief:] [deadline:] [formats:] [tech:] [references:] [deliverables:] [revisions:]	Group leader (own department) task.edit	Change a task
/task withdraw task: [reason:]	Group leader (own department) task.offer	Take back an offer that has not been answered
/task mine	Anyone	Your offers and current assignments

/time **Anyone on the team**

Local times for staff, using each person's own timezone

COMMAND	WHO CAN USE IT	WHAT IT DOES
/time member member:	Anyone	Show a staff member's current local date and time
/time department [department:]	Anyone	Show local times across a department

/trial **Anyone on the team** – **Owner**

Trial briefs for people the studio is considering

COMMAND	WHO CAN USE IT	WHAT IT DOES
/trial offer person: title: brief: terms: [department:] [pay:] [currency:] [deadline:]	Owner staff.manage	Write a trial brief and send it
/trial submit code: links: [note:]	Whoever the trial belongs to	Submit your trial work <i>A trial brief is submitted by the person on trial and nobody else.</i>
/trial decide code: outcome: feedback:	Owner staff.manage	Pass or fail a submitted trial, with feedback
/trial withdraw code: reason:	Owner staff.manage	Withdraw a trial that has not been decided
/trial list [status:]	Owner staff.manage	Trials, open by default
/trial mine	Anyone	Your own trial briefs

/web Owner

The studio website: services, pages, client logins and publishing

COMMAND	WHO CAN USE IT	WHAT IT DOES
/web identity [name:] [tagline:]	Owner config.manage	Your studio's name and tagline, as the site shows them
/web service key: name: summary: [detail:] [order:]	Owner config.manage	Write or rewrite one service on the public site
/web publish-service key: published:	Owner config.manage	Show or hide a service on the public site
/web page key: title: body:	Owner config.manage	Write a page, such as About
/web publish-page key: published:	Owner config.manage	Show or hide a page
/web status	Owner config.manage	What the public site currently shows
/web client-email client: email: [can-approve:]	Owner config.manage	Let a client sign in to the website with an email address
/web revoke-email client: email:	Owner config.manage	Remove an email address's access
/web sign-in-link client: email:	Owner config.manage	Make a one-time sign-in link to send a client yourself

/who-is-free Group leader

Who has room for more work, and who has not

COMMAND	WHO CAN USE IT	WHAT IT DOES
/who-is-free [days:] [group:]	Leader or owner	Who has room for more work, and who has not <i>A leader sees the departments they lead; the owner sees the studio.</i>